Spatial Modelling (4EK662)
Prostorové modely (4EK462)

Spatial Modelling

Block 2 – Spatial networks

Tomáš Formánek

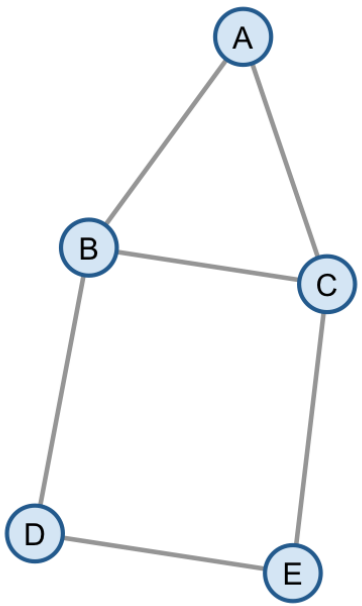
Block 2: Spatial networks (TOC)

1. Introduction to graph theory
2. Urbanistics & Space syntax analysis of urban areas
3. Transportation-related applications

Introduction to graph theory

Graph is a structure consisting of a non-empty set of **vertices** (nodes) and **edges** (arcs, links, branches):

$$G = (V, E)$$



Adjacent vertices: two vertices are adjacent, if there is an edge between them.

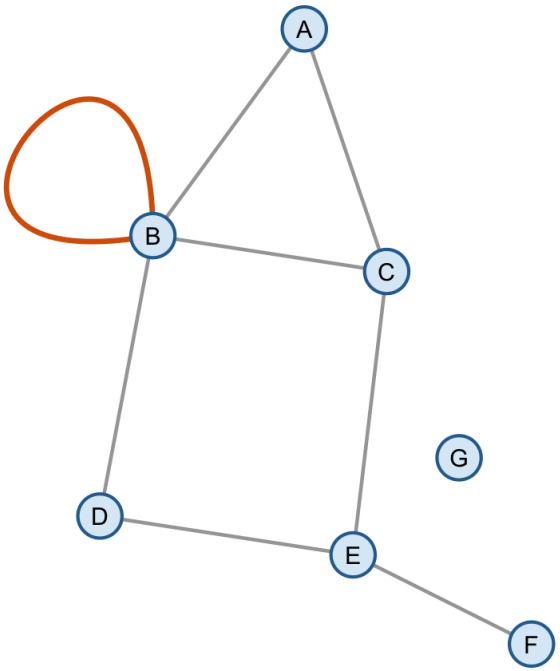
Degree of a vertex v: $d(v)$ is the number of edges incident to it.

Graph example:

V: $\{A, B, C, D, E\}$

E: $\{\{A, B\}, \{A, C\}, \{B, C\}, \{B, D\}, \dots\}$

$d(A) = 2, \quad d(B) = 3, \ \dots$



Loop: an edge that connects a vertex to itself.

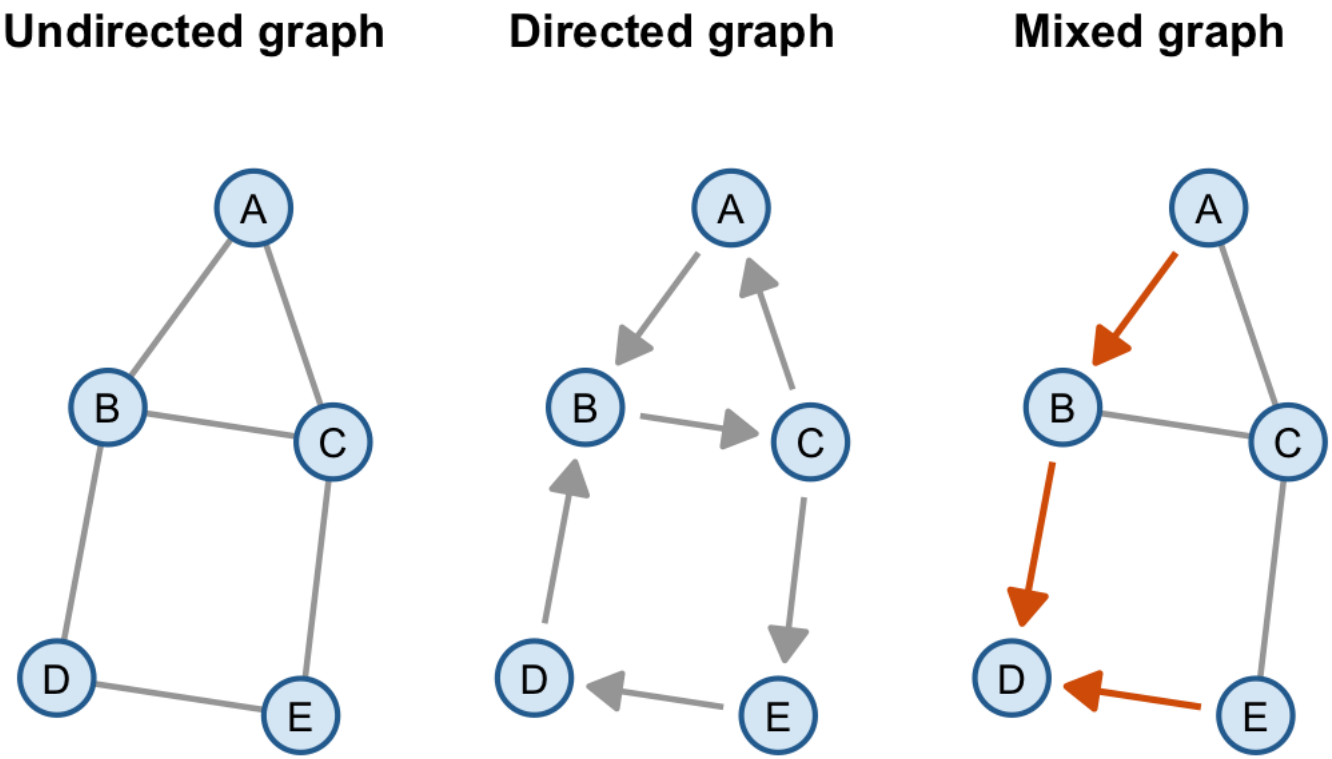
Pendant vertex: if and only if it has degree one (only 1 edge is incident to it).

Isolated vertex (island): vertex of degree zero (no incident edges, no adjacent vertices).

v	A	B	C	D	E	F	G
$d(v)$	2	5	3	2	3	1	0

Note: Loop (in an undirected graph) has two ends, both are incident to the same node. Hence the loop contributes 2 to $d(B)$

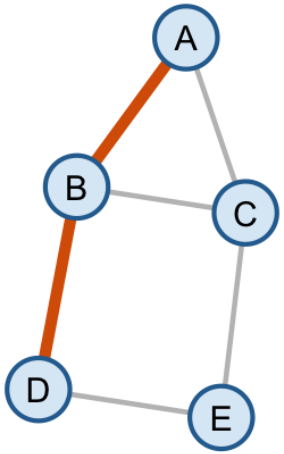
Introduction to graph theory



- **Undirected graph** $G = (V, E)$ is a non-empty set of vertices and edges. Edge connects its endpoints (loops may be present).
- **Directed graph:** non-empty set of vertices and directed edges. Directed edge is associated with an ordered pair of vertices. An edge $e = \{u, v\}$ is said to start at u and end at v.
- **Mixed graph:** contains both directed and undirected edges.

Introduction to graph theory – Path

Undirected graph



Path: connects the starting vertex v_1 to an ending vertex v_k through a continuous set of edges.

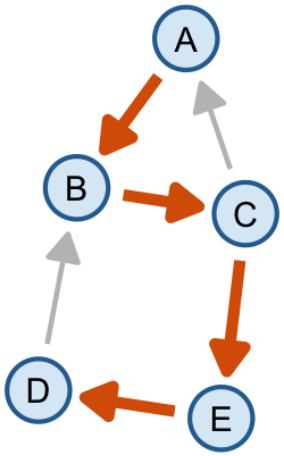
Path cannot repeat any vertex or edge. For a path going through n separate verices $(v_1, v_2, \ldots, v_n)$, a total of $n-1$ distinct edges are used.
(**Trail** allows repeated vertices, **Walk** allows both)

Length of a path: the number of edges it contains.

Shortest path: will be discussed separately.

Directed path: a path on a directed graph, where all edges point in the same forward direction.

Directed graph



Cycle: an empirically important path-like sequence where only the initial and final vertices are identical.
Circuit: a closed trail: starts and ends at the same vertex, no edge is repeated, intermediate vertices may be repeated (*cycle and cirucit are not paths*).

Note: definitions may vary across literature.

Euler trail

Euler trail is a trail that traverses **every edge of the graph exactly once**. Vertices may be visited more than once, but edges may not be repeated.

Euler circuit

Euler circuit is an Euler trail that starts and ends at the same vertex. Can revisit intermediate vertices. Every Euler circuit is also an Euler trail.

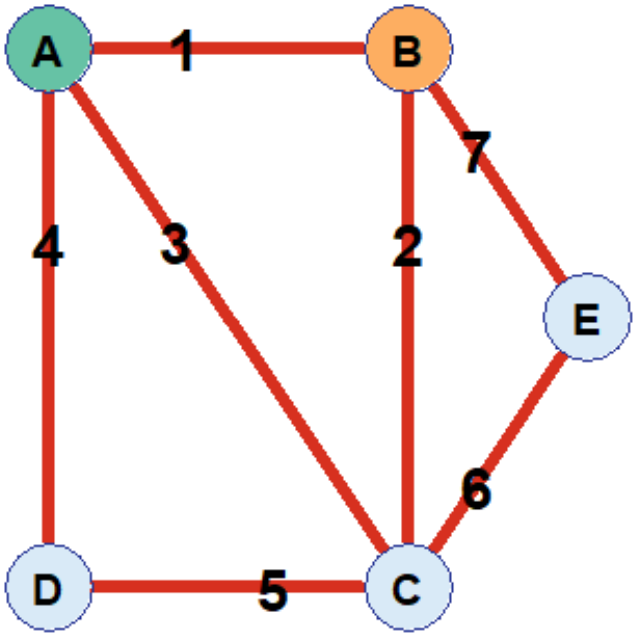
For a connected undirected graph:

$$\begin{aligned} \text{all vertices have even degree} &\iff \text{an Euler circuit exists,} \\ \text{exactly two vertices have odd degree} &\iff \text{an (open) Euler trail exists.} \end{aligned}$$

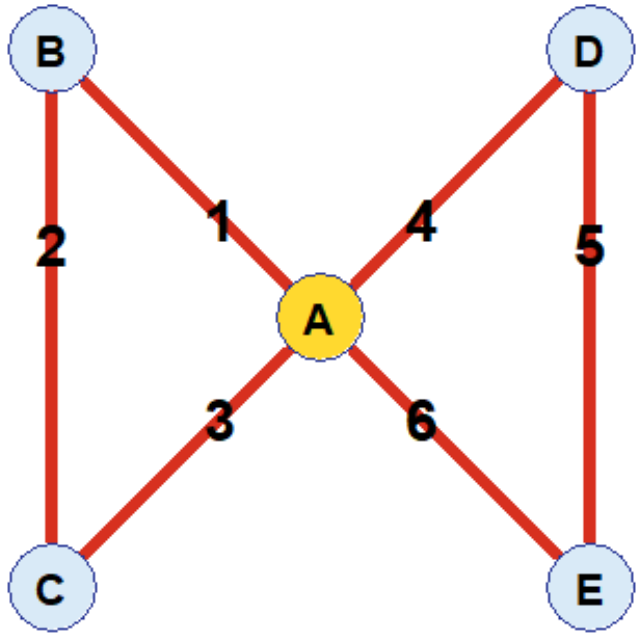
Euler: traverse every edge exactly once

Introduction to graph theory – Euler trails and circuits

Euler trail



Euler circuit



- **Euler trail:** The graph has exactly two odd-degree vertices, A and B. Consequently, it has an open Euler trail but no Euler circuit.
- **Euler circuit:** all degrees are even.

Hamilton path

Hamilton path is a path that visits **every vertex of the graph exactly once**. It does not need to use every edge of the graph.

Hamilton cycle

A **Hamilton cycle** is a Hamilton path-like sequence that returns to its starting vertex. Only the starting vertex is repeated, only to close the cycle.

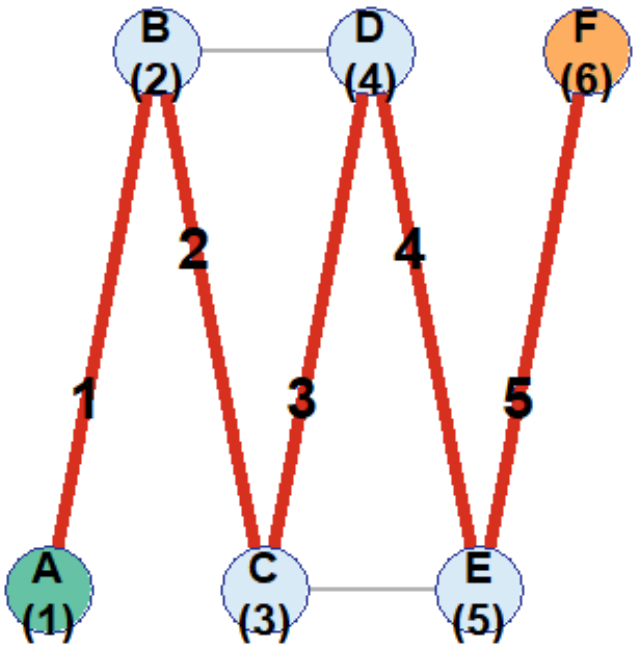
For a graph with n vertices:

$$\begin{aligned} \text{Hamilton path} &: v_1, v_2, \ldots, v_n, \\ \text{Hamilton cycle} &: v_1, v_2, \ldots, v_n, v_1. \end{aligned}$$

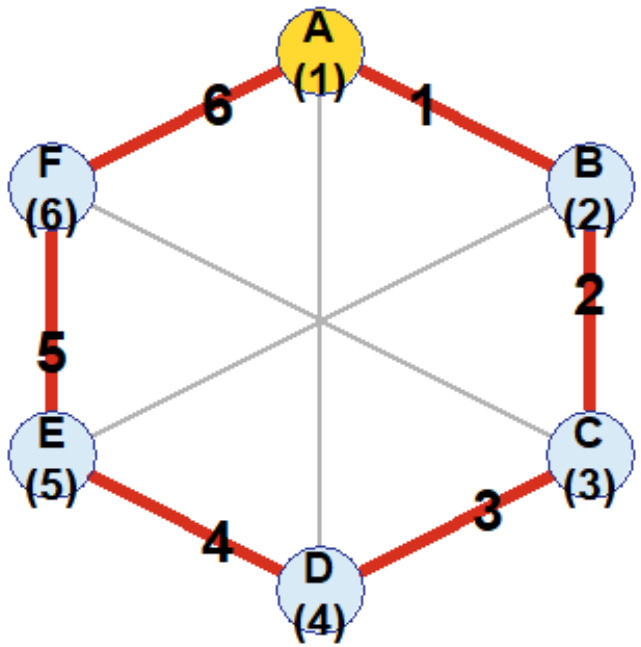
Hamilton: visit every vertex exactly once

Euler problems concern edges, can be solved efficiently using degree conditions. Hamiltonian problems concern vertices and generally require a combinatorial search.

Hamilton path



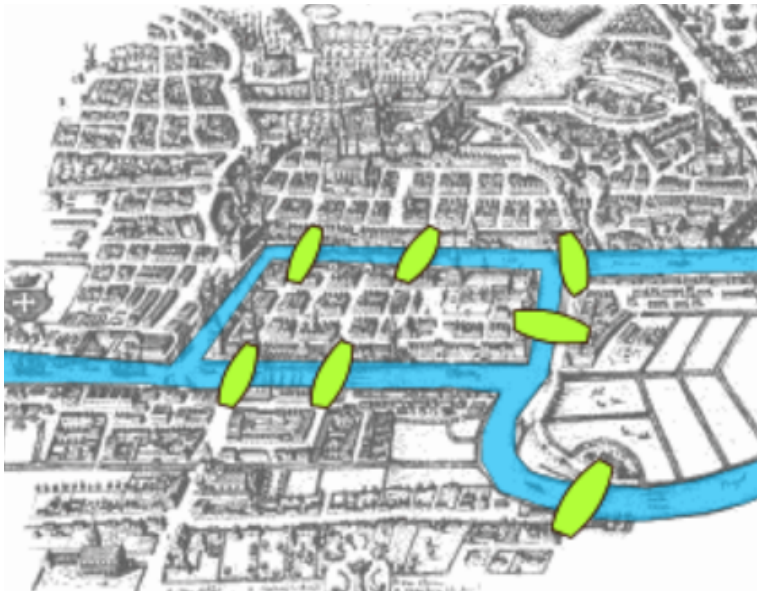
Hamilton cycle



- **Hamilton path:** $d(A) = d(F) = 1$, which makes a Hamilton cycle impossible: a vertex on a cycle needs two incident cycle edges.
- **Hamilton cycle:** the outer hexagon forms a Hamilton cycle, internal edges show that a Hamilton cycle does not have to use every edge.

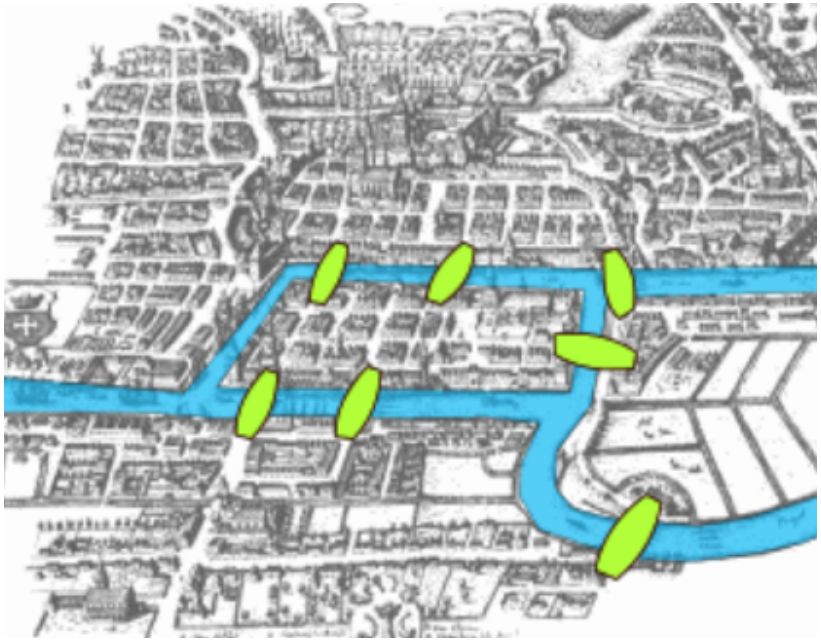
Introduction to graph theory – Königsberg Bridges

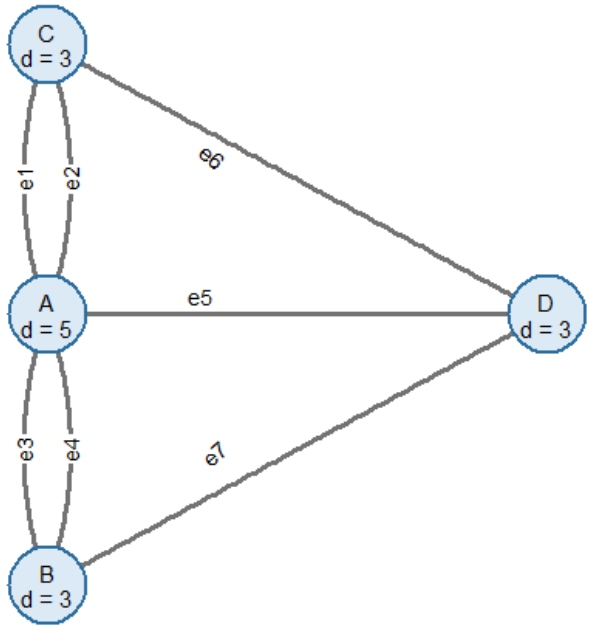
The town of Königsberg (Prussia), now Kaliningrad (Russia) was divided into four sections (nodes) by the branches of the Pregel river. In the 18th century, seven bridges connected the sections.



Problem of the Konigsberg Bridges: Is there a trail ("path") that crosses each bridge exactly once and returns to the starting point?

Introduction to graph theory – Königsberg Bridges

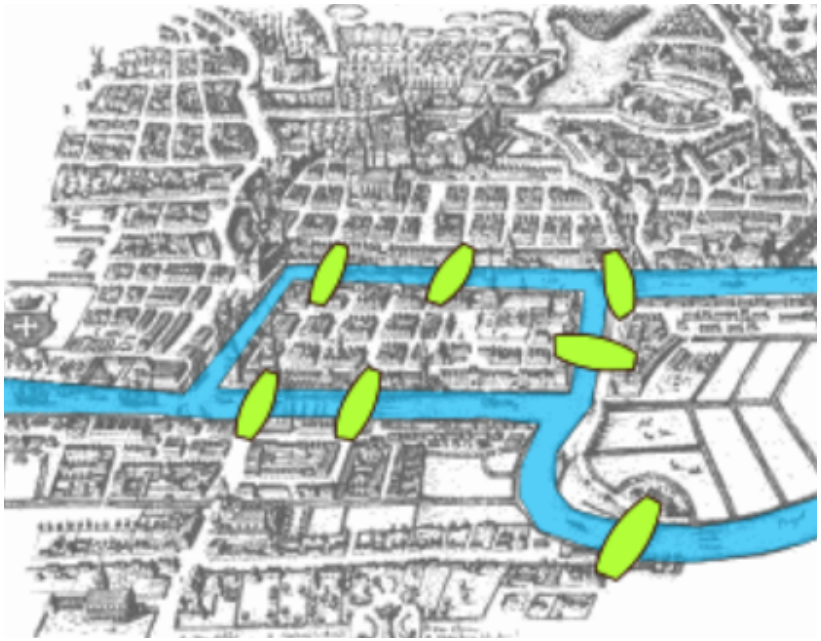


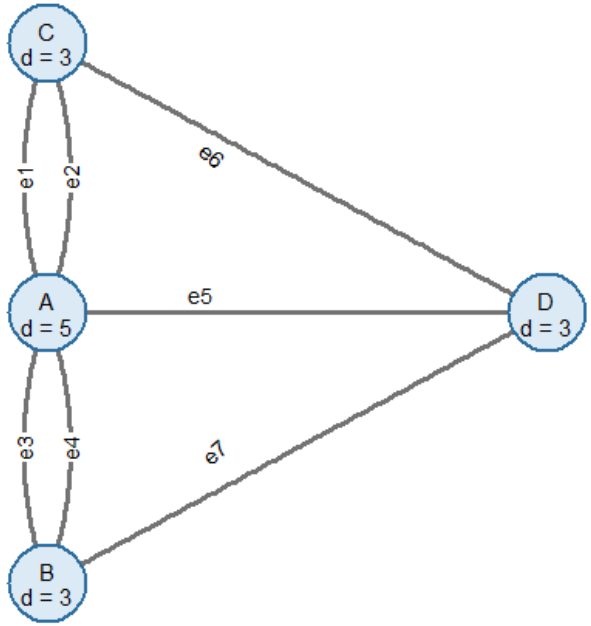


Problem of the Konigsberg Bridges: Is there a trail ("path") that crosses each bridge exactly once and returns to the starting point?

Euler or Hamilton ?

Introduction to graph theory – Königsberg Bridges



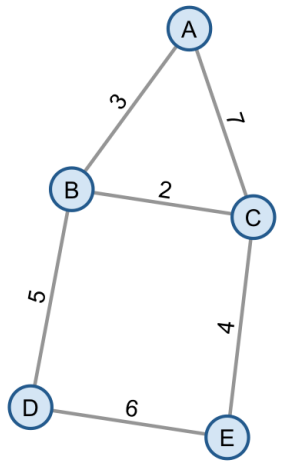


- Problem of the Konigsberg Bridges: does Euler circuit exist?
- Euler circuit exists if every vertex has even degree.
- Open Euler trail exists if exactly two vertices have an odd degree.
- The Königsberg graph has four odd-degree vertices: **no Euler path**.

- The graph has a Hamilton path & cycle. However, this does **not** solve the Königsberg bridges problem (4 bridges used, 3 left unvisited).

Introduction to graph theory – Weighted graph

Weighted graph: a graph equipped with a weight function that assigns a numerical value to every edge (node weights can also be defined and used).



Weighted graph: $\boxed{G_w = (V, E, w)}$

where $w : E \to \mathbb{R}$ is a weight function assigning a numerical value $w(e)$ to each edge $e \in E$.

Undirected graph: $w(uv) = w(vu)$.

For a directed graph, edges are ordered pairs (u, v), and in general $w(uv) \neq w(vu)$.

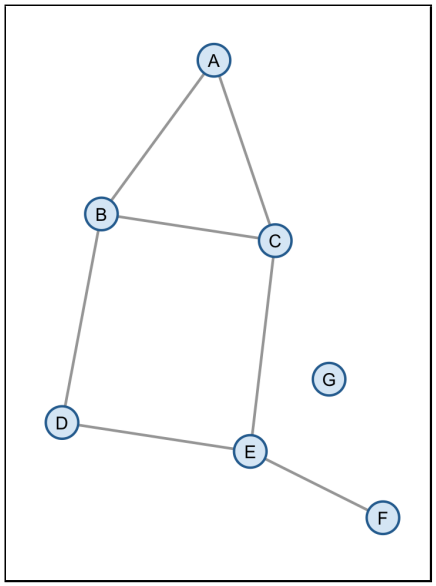
An unweighted graph may be viewed as a weighted graph with $w(e) = 1$ for all $e \in E$.

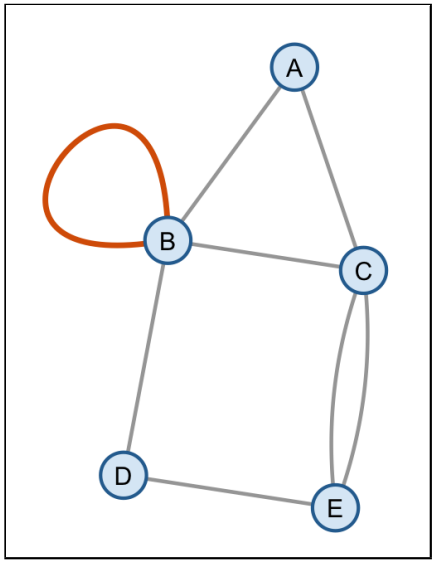
Weight of a path $P = (v_1, v_2, \ldots, v_n)$ is the sum of weights of constituent edges:

$$w(P) = \sum_{i=2}^{n} w(v_{i-1}, v_i)$$

Introduction to graph theory – Simple graphs

- **Simple graph:** is an undirected graph with no self-loops and no multiple edges between the same pair of vertices (definition does not exclude pendant and isolated vertices).





- **Multi-graph / Pseudo-graph:** extends simple graph definition by allowing multiple (parallel) edges between the same pair of vertices, allowing for loops, and directed edges (multi-graph vs pseudo-graph: definitions may vary in literature).

Introduction to graph theory – Types of simple graphs

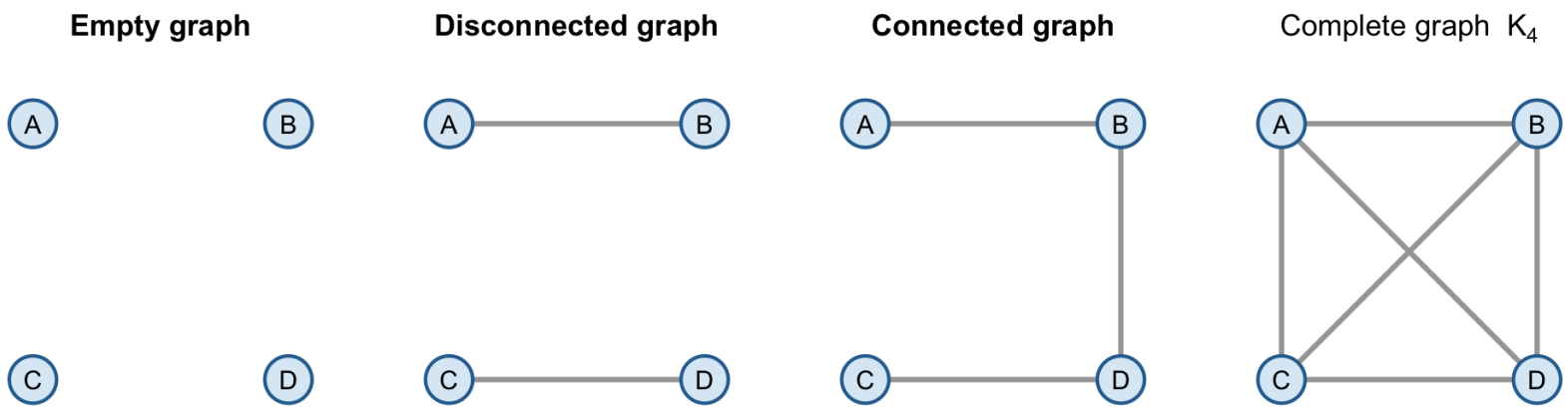
Simple graphs can be organized by:

- Connectivity
- Basic named graph families
- Based on vertex degree
- Based on cycles
- Based on planar representation

Graph types are not mutually exclusive. For example, a graph can be connected, regular, and planar.

Connectivity-based taxonomy

- **Null/empty graph:** has vertices but no edges, all vertices are isolated.
- **Disconnected graph:** at least one pair of vertices has no connecting path. It consists of two or more connected components.
- **Connected graph:** there is a path between every pair of vertices.
- **Complete graph:** every pair of distinct vertices is connected by an edge. With n vertices: $d(v_i) = n - 1\ \forall i$, and No. of edges $= n(n-1)/2$ (undirected graph).



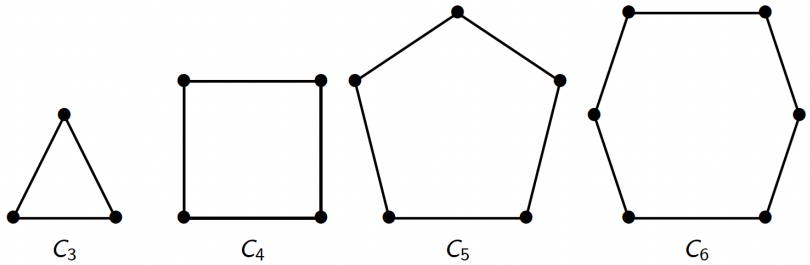
Basic named graph families

- Path graph P_n

  A sequence of n vertices connected by $(n-1)$ edges.
  A connected graph with no cycles.

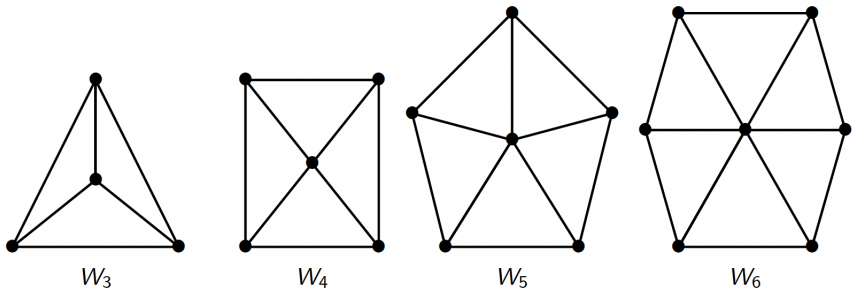
- Cycle graph C_n

  A graph in which the vertices form one cycle: $v_1 - v_2 - \cdots - v_n - v_1$.
  Every vertex has degree 2.



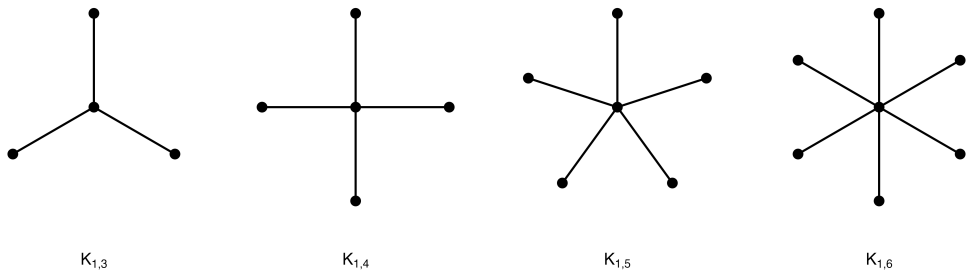
- Wheel graph W_n

  A cycle with one additional central vertex connected to all vertices of the cycle.

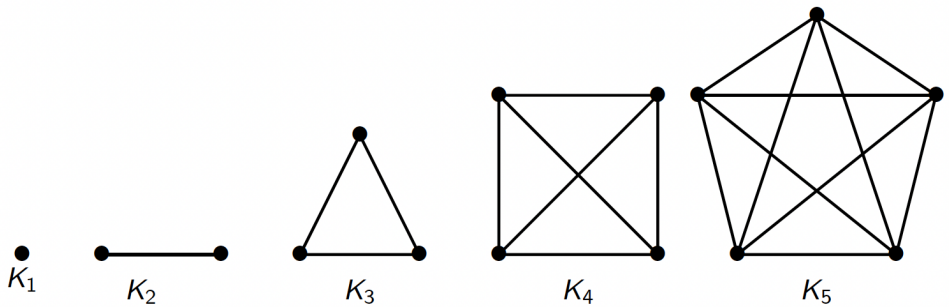


Basic named graph families

- Star graph S_n, (often denoted $K_{1,n}$, as it is bipartite)
  One central vertex is connected to all other vertices, while the remaining vertices are not mutually connected.



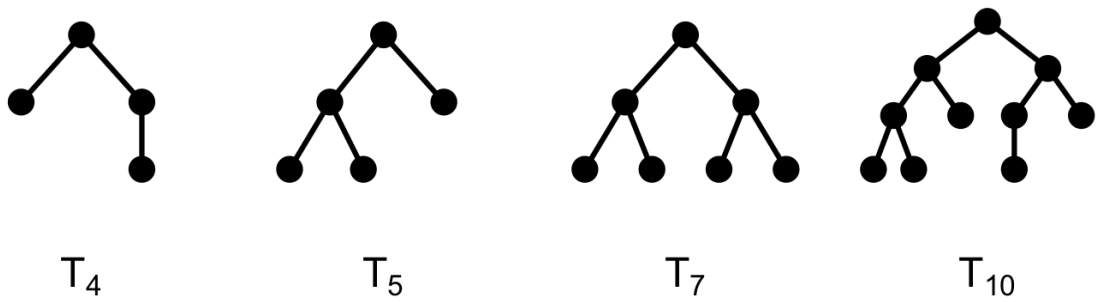
- Complete graph K_n
  With n vertices: $d(v_i) = n - 1 \; \forall i$, and No. of edges $= n(n-1)/2$.



Basic named graph families

- Tree graph

  A connected acyclic simple graph. For a tree with n vertices, the number of edges is: $|E| = n - 1$. There is exactly one path between every pair of vertices.



- Forest

  An acyclic simple graph that need not be connected. Each connected component of a forest is a tree.

Graph types based on vertex degree

- **Regular graph:** (k-regular) if every vertex has the same degree k:

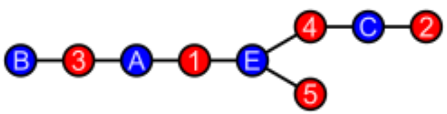
$$d(v_i) = k, \quad \forall i, \quad i = 1, \ldots, n$$

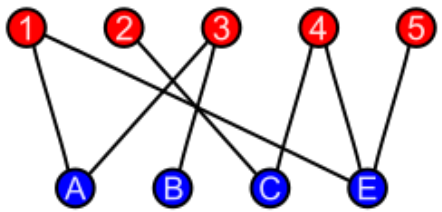
For example: Null graph: $d(v) = 0$, C_n: $d(v) = 2$, K_n: $d(v) = n - 1$.

- **Bipartite graph:** A graph $G = (V, E)$ is bipartite if its vertex set can be divided into two disjoint subsets/groups:

$$V = V_1 \cup V_2, \qquad V_1 \cap V_2 = \emptyset.$$

Every edge has one endpoint in $V1$ and the other endpoint in $V2$. No withing-group edges (S_n is bipartite, C_n is bipartite for even n).





Graph types based on cycles

- **Acyclic graph:** contains no cycles (e.g. tree, forest)
- **Cyclic graph:** contains at least one cycle.
- **Unicyclic graph:** connected and contains exactly one cycle.

Note that "cycle graph" and "cyclic graph" are not synonymous:

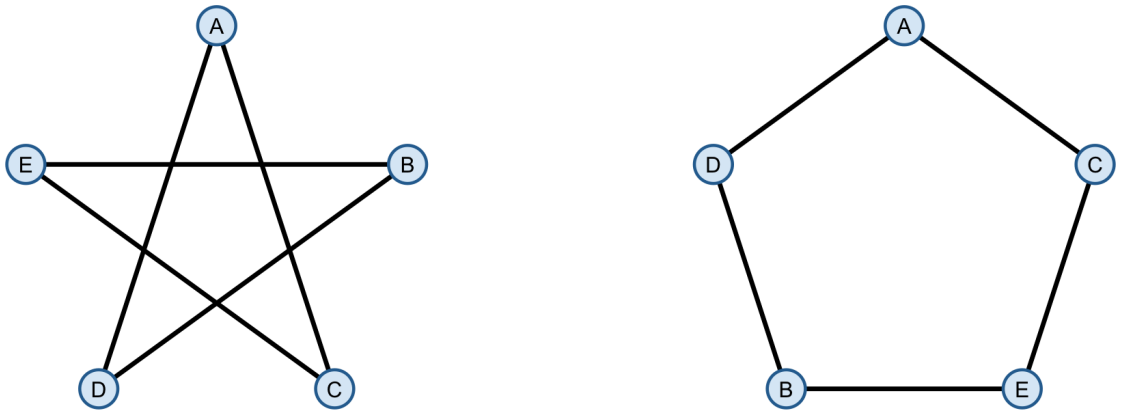
- C_n is one specific cycle
- cyclic graph is any graph containing at least one cycle

Graph types based on planar representation

- Planar graph

  A graph that can be drawn in the plane without any edges crossing (edges can meet at a shared vertex).

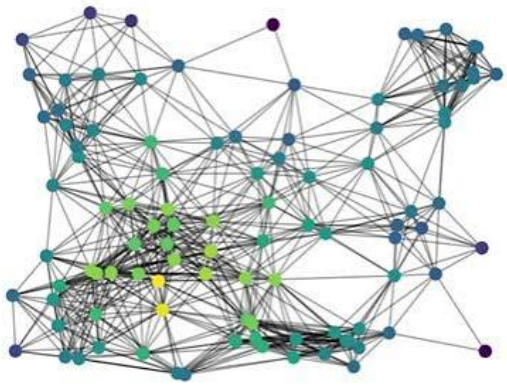
  Planarity is a property of the graph itself, not of one particular drawing. A convenient drawing may contain crossings even though the graph is planar.



Two different geometric drawings of the same labeled graph.
Planar graph: at least one drawing without intersecting edges is possible.

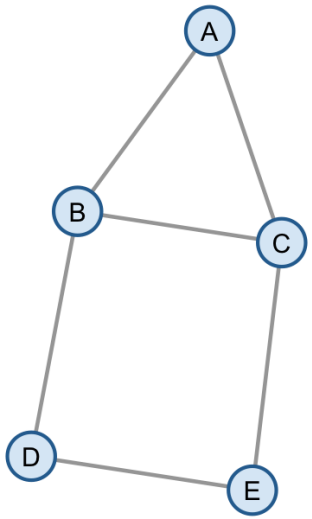
Introduction to graph theory – Graph representation

- **Adjacency list:** for each vertex, lists all adjacent vertices.
- **Adjacency matrix:** a $|V| \times |V|$ matrix indicating which pairs of vertices are adjacent.
- **Incidence matrix:** a $|V| \times |E|$ matrix indicating which vertices are incident to which edges.



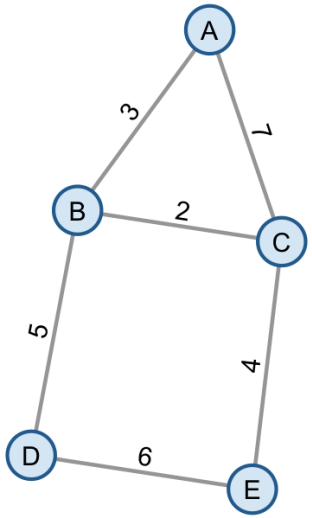
- **Set of two tidy tables**: one for vertices, one for edges (`R, tidyverse`).

Introduction to graph theory – Adjacency list



Adjacency list

Vertex	: Neighbors
A	$: B, C$
B	$: A, C, D$
C	$: A, B, E$
D	$: B, E$
E	$: C, D$



Adjacency list (weighted)

$A : (B, 3),\ (C, 7),$

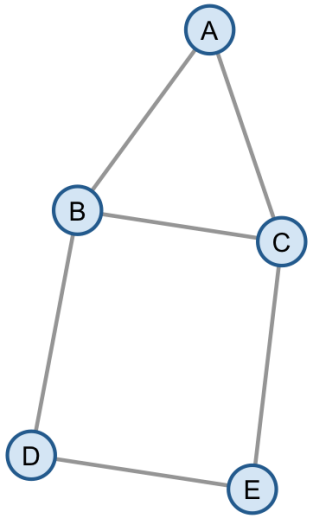
$B : (A, 3),\ (C, 2),\ (D, 5),$

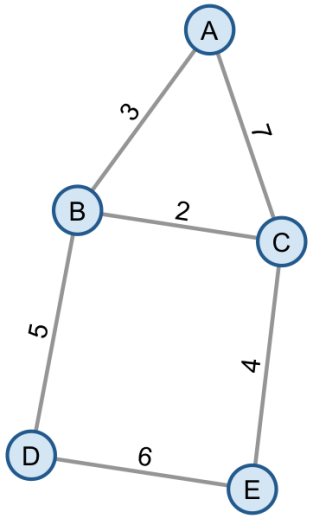
$C : (A, 7),\ (B, 2),\ (E, 4),$

$D : (B, 5),\ (E, 6),$

$E : (C, 4),\ (D, 6).$

Each edge appears twice in the adjacency list (each endpoint, undirected).





Weighted adjacency matrix

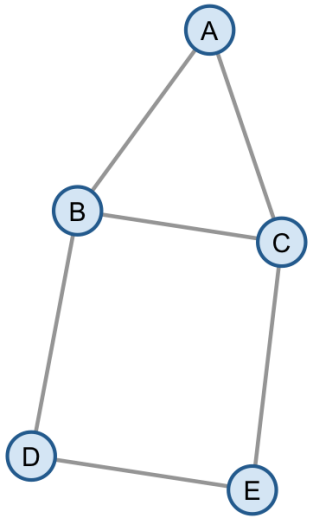
$$\mathbf{W} = \begin{array}{c|ccccc} & A & B & C & D & E \\ \hline A & 0 & 3 & 7 & 0 & 0 \\ B & 3 & 0 & 2 & 5 & 0 \\ C & 7 & 2 & 0 & 0 & 4 \\ D & 0 & 5 & 0 & 0 & 6 \\ E & 0 & 0 & 4 & 6 & 0 \end{array}$$

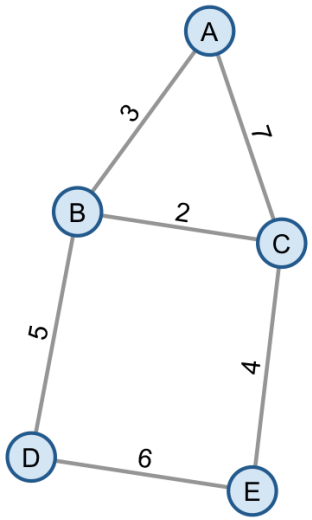
For unweighted graphs: $w_{ij} = \begin{cases} 1, & \text{if } \{v_i, v_j\} \in E, \\ 0, & \text{otherwise.} \end{cases}$

For undirected graphs: $\boldsymbol{W} = \boldsymbol{W}^\intercal$, i.e. $w_{ij} = w_{ji}$.

Shortest-path calc.: $w_{ij} = \begin{cases} \text{dist}_{ij}, & \text{if } \{v_i, v_j\} \in E, \\ \infty, & \text{otherwise.} \end{cases}$

Introduction to graph theory – Incidence matrix





We start by labelling the edges:

$$e_1 = \{A, B\}, \quad e_2 = \{A, C\}, \quad e_3 = \{B, C\},$$
$$e_4 = \{B, D\}, \quad e_5 = \{C, E\}, \quad e_6 = \{D, E\}.$$

Incidence matrix

$$\boldsymbol{B} = \begin{array}{c|cccccc} & e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \\ \hline A & 1 & 1 & 0 & 0 & 0 & 0 \\ B & 1 & 0 & 1 & 1 & 0 & 0 \\ C & 0 & 1 & 1 & 0 & 1 & 0 \\ D & 0 & 0 & 0 & 1 & 0 & 1 \\ E & 0 & 0 & 0 & 0 & 1 & 1 \end{array}, \quad \boldsymbol{w} = \begin{bmatrix} 3 \\ 7 \\ 2 \\ 5 \\ 4 \\ 6 \end{bmatrix}.$$

$\boldsymbol{B}$ describes graph structure,
$\boldsymbol{w}$ describes edge values.

Directed graph: $b_{ij} = -1$ if edge leaves a node,
$b_{ij} = 1$ if edge enters.
$A \to B$: $b_{11} = -1$, $b_{21} = 1$.

Graphs in R – tidy tables

Tidy table (definition by H. Wickham of the `tidyverse` community):

1. Each variable forms a column
2. Each observation forms a row
3. Each value occupies a single cell

A graph $G = (V, E)$ is not a tidy table object, as it has two separate sets of objects: V, and E. `tidygraph` represents G as two linked tidy tables:

Node table

node	population
A	100
B	50
C	75

Edge table

from	to	distance
A	B	10
A	C	25
B	C	15

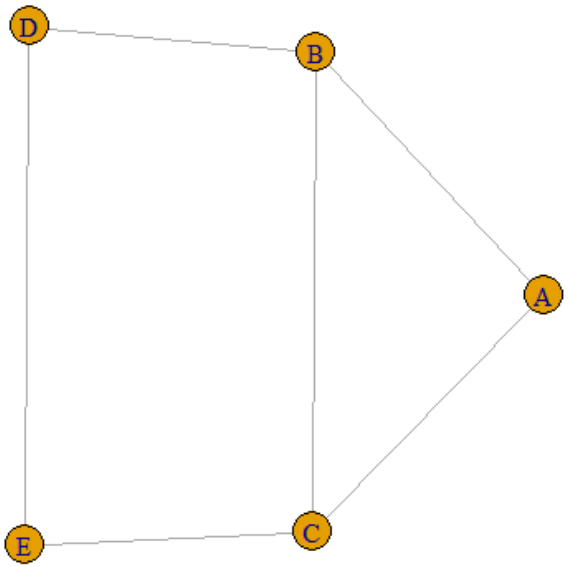
Note on flexibility: here, values can be assigned to both nodes and edges in an easy-to-access manner.

Graphs in R

Packages for graph handling and analysis:

- `igraph`: a fast open source platform for the analysis of graphs or networks. Core written in C, packages exist for `R`, Python. `https://r.igraph.org/articles/igraph.html`
- `tidygraph`: a tidy (tidyverse) interface to `igraph`, with a simple way to switch between the two tidy tables (nodes, edges): `activate("nodes")`, `activate("edges")`; it also provides `dplyr` verbs for manipulating the tables. `https://tidygraph.data-imaginist.com/`
- `sfnetworks`: connects the functionalities of the `tidygraph` package for network analysis and the `sf` package for spatial data science. `https://luukvdmeer.github.io/sfnetworks/`
- `ggraph`: extends `ggplot2`, aimed at supporting relational data structures such as networks, graphs, and trees. `https://ggraph.data-imaginist.com/`

Graphs in R – igraph package




```
> g1 <- igraph::make_graph(
>     ~ A--B,
>     A--C,
>     B--C,
>     B--D,
>     C--E,
>     D--E)

>g1
| IGRAPH 5dcd7ea UN-- 5 6 --
| attr: name (v/c)
| edges from 5dcd7ea (vertex names):
| [1] A--B A--C B--C B--D C--E D--E

# U N - -
# | | | |
# | | | --- bipartite status (B/-)
# | | ----- weighted status (W/-)
# | ------- named vertices
# --------- directed or undirected

# 5 6 <- 5 vertices, 6 edges

> plot(g1)
# igraph function, not from the ggraph package
```

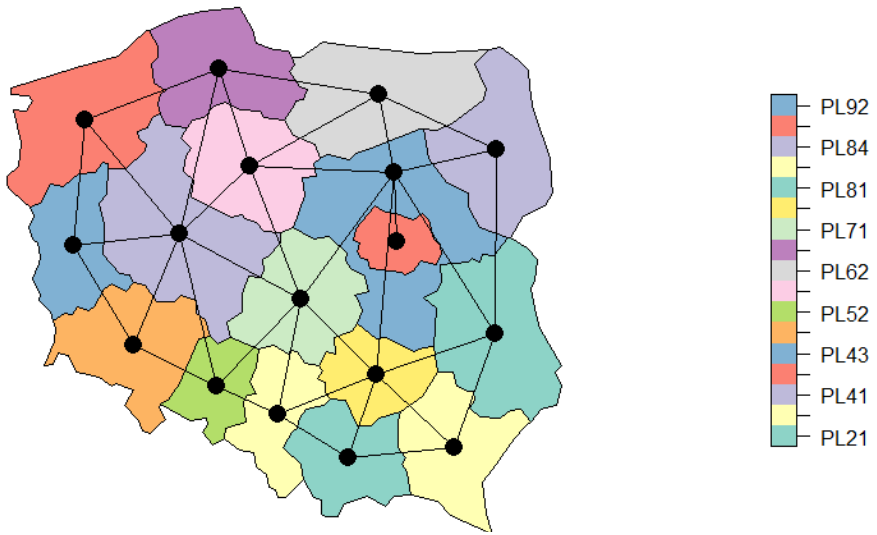
Graphs in R: Regional connectivity example

Packages used: sf, spdep

```
# Poland.sf is a sf dataframe with 17 NUTS2 polygons of Poland
> PL_contig <- spdep::poly2nb(Poland.sf)
> PL_contig
| Neighbour list object:
| Number of regions: 17
| Number of nonzero links: 70
| Percentage nonzero weights: 24.22145
| Average number of links: 4.117647
# Adjacency list for the first region (PL21)

> PL_contig[[1]]
| [1]  2 12 14

# for the plot below, centroids must also be created (to generate NUTS2-representations/nodes)
```



Graphs in R: Regional connectivity example

Connectivity matrix: contiguity-based (common border), NUTS 2, Poland.

```
> C <- nb2mat(PL_contig, style="B") # nb to binary matrix of connectivity
> rownames(C) <- Poland.sf$NUTS_ID
> C
```

$\boldsymbol{C} =$

PL21	0	1	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0
PL22	1	0	0	0	0	0	1	0	0	0	1	1	0	0	0	0	0
PL41	0	0	0	1	1	1	1	1	0	1	1	0	0	0	0	0	0
PL42	0	0	1	0	1	0	0	0	0	1	0	0	0	0	0	0	0
PL43	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0
PL51	0	0	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0
PL52	0	1	1	0	0	1	0	0	0	0	1	0	0	0	0	0	0
PL61	0	0	1	0	0	0	0	0	1	1	1	0	0	0	0	0	1
PL62	0	0	0	0	0	0	0	1	0	1	0	0	0	0	1	0	1
PL63	0	0	1	1	0	0	0	1	1	0	0	0	0	0	0	0	0
PL71	0	1	1	0	0	0	1	1	0	0	0	1	0	0	0	0	1
PL72	1	1	0	0	0	0	0	0	0	0	1	0	1	1	0	0	1
PL81	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0	1
PL82	1	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
PL84	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	1
PL91	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
PL92	0	0	0	0	0	0	0	1	1	0	1	1	1	0	1	1	0

Urbanistics & Space syntax analysis of urban areas

Urbanistics

Urbanistics: interdisciplinary study of cities, urban spaces, and the interactions between people, infrastructure, economic activities, and the built environment.

Urbanistics combines/draws from:

- Urban planning and design
- Geography and spatial analysis
- Transportation and mobility studies
- Economics and regional science
- Sociology and environmental studies

Central questions:

- How are cities spatially organized?
- How do people move through and use urban space?
- What makes locations (un)attractive?
- How urban environment affects quality of life and sustainability?

Urbanistics

Cities and urban areas: interconnected **spatial systems** where streets, transport networks, land uses, facilities, and human activities interact. Main topics of analysis include:

- Accessibility of urban spaces
- Pedestrian and vehicle movement
- Public transport efficiency
- Street-network connectivity
- Land-use patterns and urban form
- Distribution of facilities and services
- Urban growth and spatial expansion
- Environmental quality and sustainability

→ Complex systems without easily accessible causality patterns (paradigm: city planning/store/individual; granularity; relevant time frame of the study/planning).

→ Many urban phenomena can be represented and analyzed as **spatial networks**.

Urbanistics and graph representations of urban areas

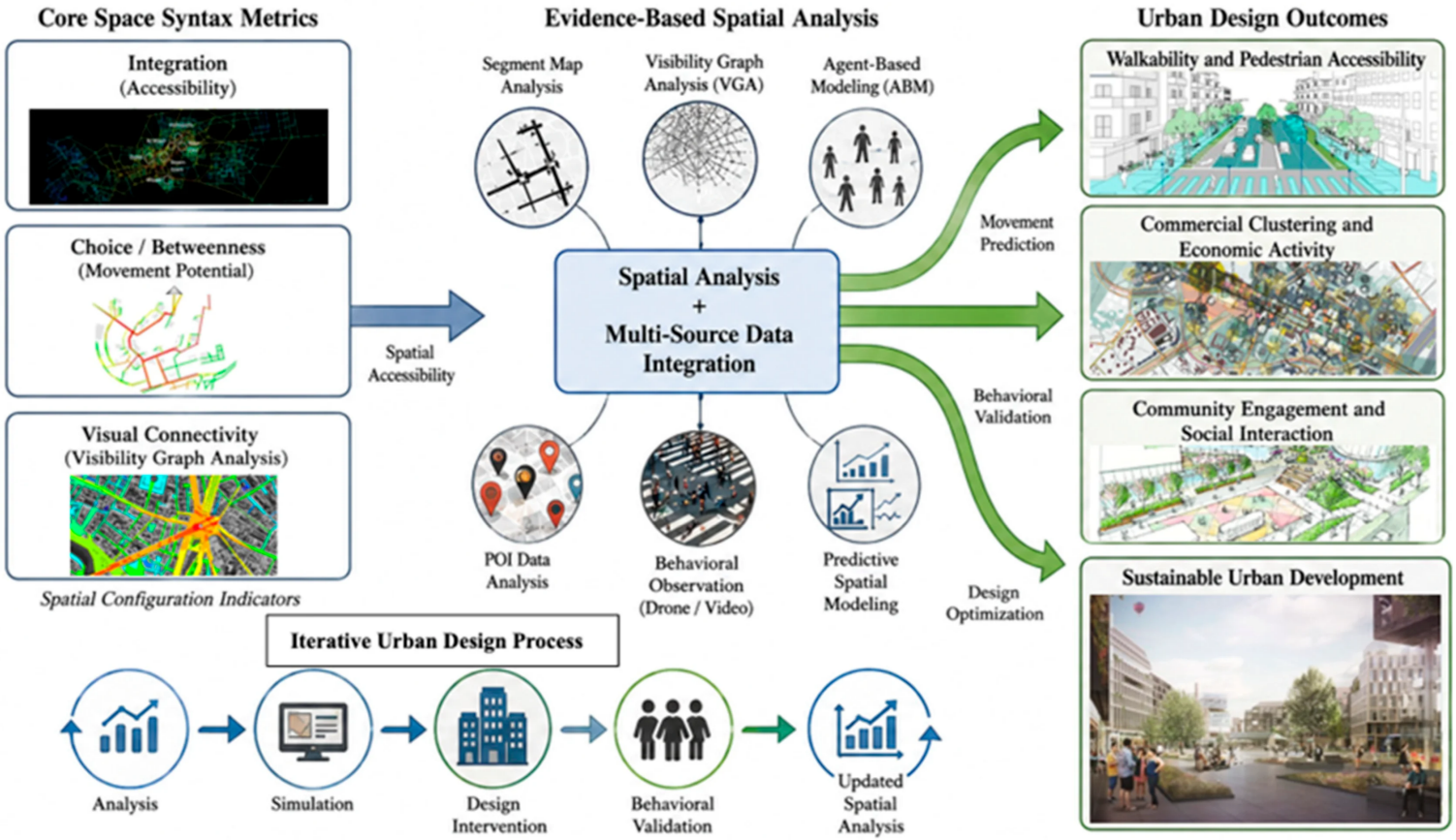


- **Axial map:** network made of the longest, fewest straight lines that cover all public streets and open spaces in a city. Each straight line represents a continuous space that a person can both see down & walk through.
- **Convex map:** Urban space is divided into the smallest set of convex spaces (building layouts, human interactions)
- **Isovist/visibility map:** Represent the area visible from a location.
- **Street/spatial network:** Connectivity analysis, routing, etc.
- **Heatmap:** Pedestrian flow (or other type of street usage intensity).

Use: network analysis, choosing optimal coverage for facility sites, travel time maps (isochrones), land use categorization, and many more.

Urbanistics – Space syntax analysis (SSA)

Spatce syntax analysis: set of quantitative techniques used to study how the layout of streets, buildings, and public spaces influences human movement, social behavior, and urban vitality.



Founders: Bill Hillier & Julienne Hanson; University College London (1970s – 1980s)

Urbanistics – Space syntax analysis (SSA)

Main quantitative metrics of space syntax analysis used for description of city locations (nodes on a graph)

- Depth
- Integration (relative asymmetry, closeness centrality)
- Choice (betweenness centrality)

Maximal Coverage Location-Allocation spatial optimization
MCLA: Assessing and planning infrastructure and service networks, given a dispersed demand, limited capacity, accessibility targets, and concerns about spatial justice.

SSA: Depth

Depth measures how far one space is from another within a spatial system. In an axial or segment graph, depth corresponds to the number of topological steps (direction changes) required to move from one space to another.

In graph theory, depth is the shortest-path distance.

For a node i, the **depth** to node j is: d_{ij} (i.e. distance from i to j).

The **total depth** of node i is: $TD_i = \sum_{j \neq i} d_{ij}$

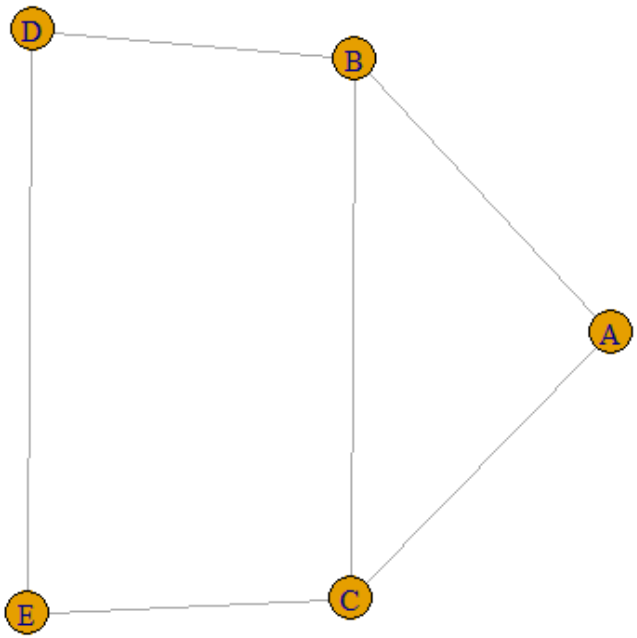
The **mean depth** of node i is: $MD_i = \dfrac{\sum_{j \neq i} d_{ij}}{n-1}, \quad i = 1, 2, \ldots, n.$

- Low depth $\rightarrow$ centrally located spaces.
- High depth $\rightarrow$ segregated or peripheral spaces.

SSA: Depth example

```
> g1 <- igraph::make_graph(
>      ~ A--B,
>      A--C,
>      B--C,
>      B--D,
>      C--E,
>      D--E)

>plot(g1)
```




```
# Depth matrix
> D <- distances(g)
> print(D)
|   A B C D E
| A 0 1 1 2 2
| B 1 0 1 1 2
| C 1 1 0 2 1
| D 2 1 2 0 1
| E 2 2 1 1 0

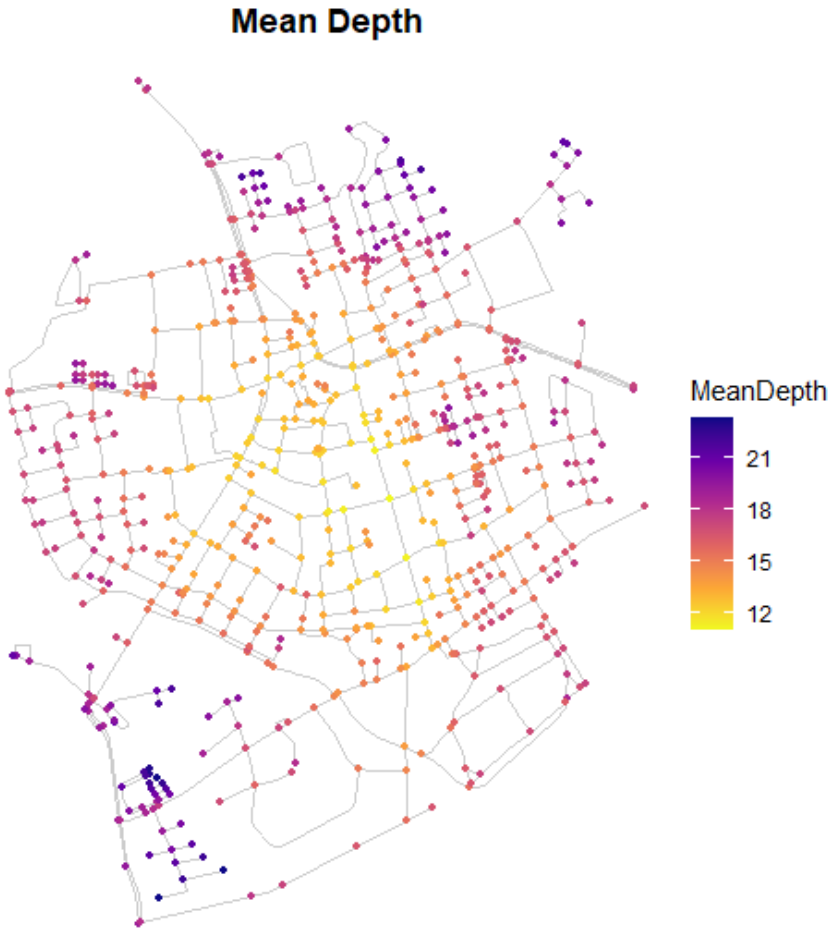
# Total Depth (TD)
> TD <- rowSums(D)
> print(TD)
| A B C D E
| 6 5 5 6 6

# Mean Depth (MD)
> n <- gorder(g)
> n
| 5
> MD <- TD / (n - 1)

> print(MD)
|    A    B    C    D    E
| 1.50 1.25 1.25 1.50 1.50
```

Depth calculation for Roxel (Münster, Germany)

The largest connected segment (sub-graph) selected for the calculation from the `roxel` sample dataset, as distances between unconnected nodes are evaluated at ∞. Low depth: centrally located, high depth: segregated or peripheral.



SSA: Integration

- **Depth:** is the fundamental quantity used for Integration analysis, depends on graph (city) size.
- **Relative asymmetry (RA):** normalizes mean depth relative to graph size. How asymmetric is a given node relative to the shallowest and deepest element in a graph?
- **Real relative asymmetry (RRA):** RA still depends on the size and structure of the graph, RRA can be used for comparison across different systems (graphs, cities).
- **Integration:** is just the inverse of RRA; the most commonly mapped and interpreted measure in space syntax analysis.

Relative asymmetry in SSA is a measure of **relative segregation**, describes how central is node i compared to the shallowest and deepest node on a graph (size-adjusted in RRA).

SSA: Relative asymmetry

Mean depth:

$$MD_i = \frac{\sum_{j \neq i} d_{ij}}{n-1},$$

where n is the number of spaces (nodes). Mean depth depends on the size of the system. To allow comparison between spaces, it is normalized using

Relative Asymmetry (RA):

$$RA_i = \frac{2(MD_i - 1)}{n-2}, \quad RA_i \in [0, 1],$$

Interpretation:

- Low $RA \rightarrow$ shallow, highly accessible space
- High $RA \rightarrow$ deep, segregated space
- $RA = 0$ represents maximum integration
- $RA = 1$ represents maximum segregation

SSA: Real relative asymmetry

RA depends on the size and structure of the graph (RA values systematically change with n). To compare RA across spatial systems, we can use

Real Relative Asymmetry (RRA):

$$RRA_i = \frac{RA_i}{D_n},$$

where

$$D_n = \frac{2\left[\left(\log_2\left(\frac{n+2}{3}\right) - 1\right) n + 1\right]}{(n-1)(n-2)},$$

is a normalization element, based on a theoretical reference graph of size n.

n	5	10	20	50	100	200
D_n	0.352	0.306	0.225	0.133	0.085	0.052

Larger systems naturally have larger depths. D_n removes this size effect. RRA and Integration values become more comparable across cities.

SSA: Integration

In SSA, Integration is defined as the inverse of RRA:

$$Integration_i = \frac{1}{RRA_i}$$

- High integration:
  - Easily reachable spaces
  - Likely movement destinations
  - Often associated with commercial activity
- Low integration:
  - Peripheral or segregated spaces
  - Less movement and visibility

SSA: Integration & Closeness centrality

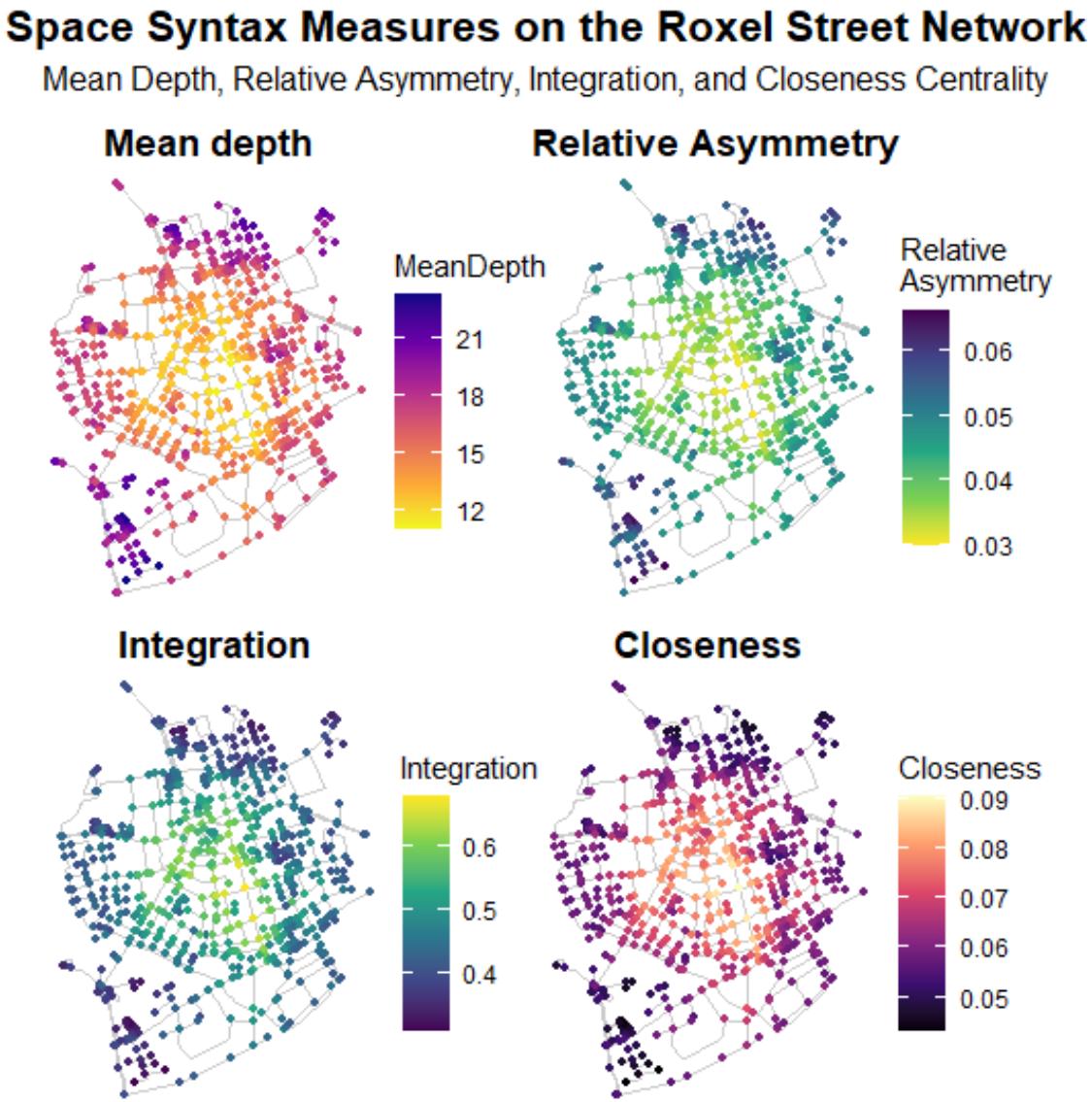
$$Integration_i = \frac{1}{RRA_i}$$

$$Closeness_i = \frac{n-1}{\sum_{j \neq i} d_{ij}} = \frac{1}{MD_i}$$

- **Global Integration/Closeness:** calculated for all $i = 1, \ldots, n$ nodes (the above formulas). How accessible is the ith location within the entire city? Used to identify city-centers (peripheries), reflects citywide movement and major retail corridors.
- **Local Integration/Closeness:** only nodes within a given radius from node i are used for calculation: $n_i(r) = \{j : d_{ij} \leq r,\ j \neq i\}$. How accessible is this location within its immediate surroundings? Used for neighborhood centers, and local pedestrian activity.

Integration is the SSA version of **Closeness centrality** metric, used in graph theory. They are not mathematically identical, but reflect the same underlying idea: accessibility through shortest paths.

SSA: Integration & Closeness centrality



Note: Scale ordering switched for Integration & Closeness for better visualization.

SSA: Choice & Betweenness centrality

$$Choice_i = Betweenness_i = \sum_{j:j \neq i \neq k} \frac{g_{jk}(i)}{g_{jk}},$$

where
g_{jk} is the number of shortest paths between graph elements j and k,
$g_{jk}(i)$ only counts shortest paths passing through i.

Captures how often i lies on paths between other elements in the city (on the graph). Through-movement potential: how likely a node is to be traversed while traveling between other places. Local version can be constructed (sensitive to boundary of the study area – edge effects) .

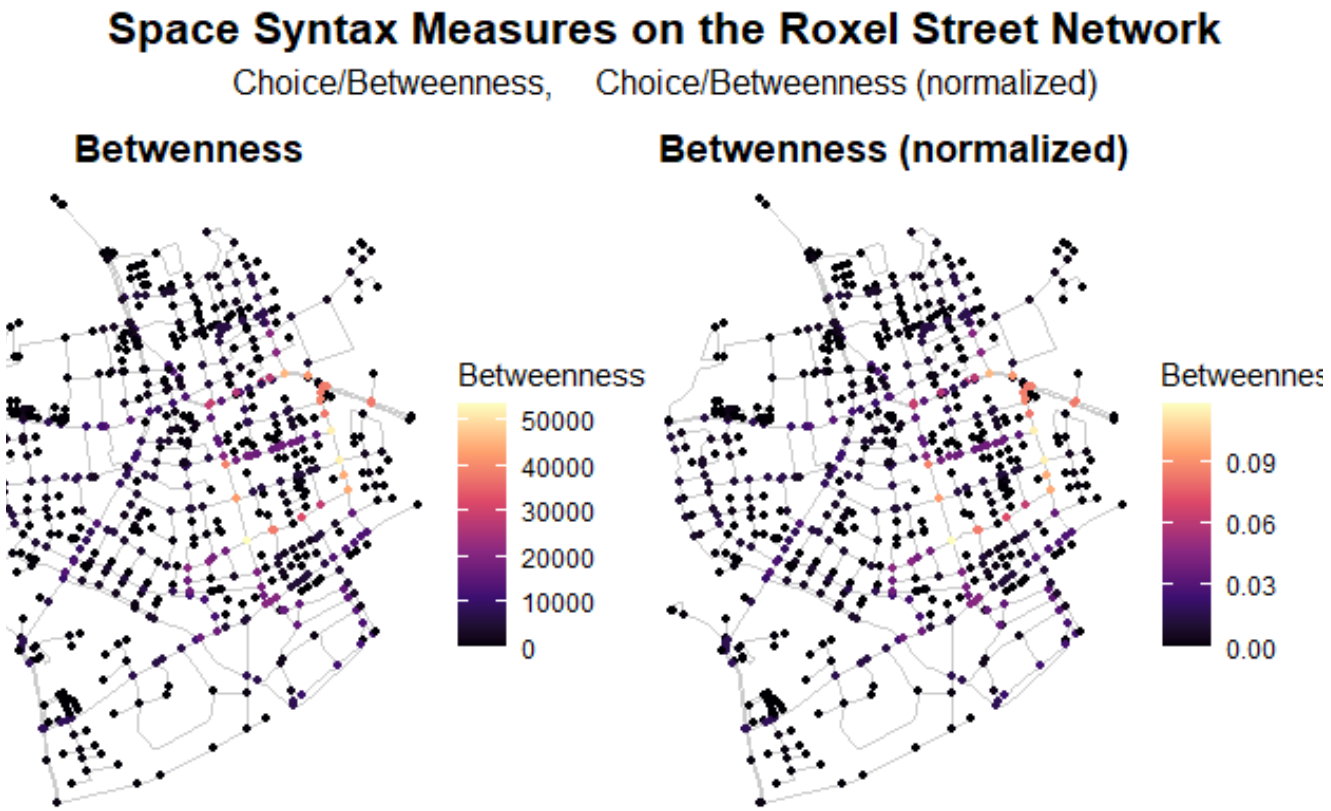
High choice:

- Main streets, bridges, major intersections
- High vehicle and/or pedestrian flow, high economic & retail activity

Low choice:

- Destination spaces (cul-de-sacs, residential streets), peripheral locations
- People may visit them, but relatively few routes pass through

SSA: Choice & Betweenness centrality



$$Betweenness_i^{(n)} = \frac{Betweenness_i}{(n-1)(n-2)}, \quad \text{is the normalized version.}$$

SSA: Depth, Integration, and Choice

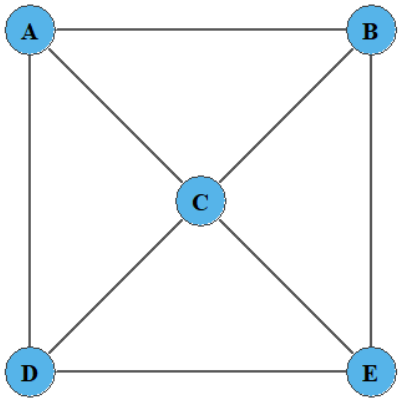
Depth The shortest-path separation between two spatial elements. Basic quantity from which the other measures are derived.

Integration Describes how close a spatial element is to all other elements. Highly integrated elements have low average depth and are well accessible from the network as a whole. (Closeness)

Choice Describes how frequently a spatial element lies on shortest or simplest paths between other pairs of elements. High choice indicates an important connector or through-route. (Betweenness)

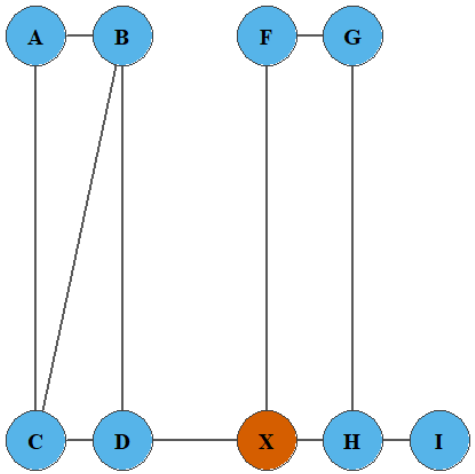
SSA: Depth, Integration, and Choice – Examples

Dense Central Structure



Node **C** has high Integration because it is directly connected to all other nodes. However, perimeter routes mean that shortest paths between other nodes do not depend exclusively on **C**. Integration/Closeness can be high without Choice/Betweenness of **C** dominating to the same degree.

Bridge Between Two Areas



D—**X** is a bridge, **D**, **X** are articulation vertices (removing them disconnects the graph). Both nodes have high Choice/Betwenness (not necessarily the same values, as they have different local connections). Paths between the left and right parts of the graph must use the bridge. **D**, **X** Integration values do not necessarily stand out.

Node **I**: cul-de-sac endpoint on a city periphery: high Depth & low Integration, zero Choice (endpoint rather than an intermediate route element)

SSA: Depth, Integration, and Choice – Applications

Depth, Integration/Closeness

- Identifying city-level and neighborhood centers
- Evaluating facility locations: placed in accessible areas?
- Comparing design alternatives: would a new connection increases or decreases accessibility?
- Identifying segregated areas

Choice/Betweenness

- Identifying principal movement corridors
- Detecting bridges and bottlenecks, traffic resilience
- Street hierarchy: citywide connectors vs. neighborhood-scale routes
- Testing new connections: how a bridge, underpass, pedestrian link, or street extension redistributes potential routes

MCLA spatial optimization

Location–allocation analysis studies the spatial placement of facilities and the assignment of spatially distributed demand to those facilities. Facility locations are evaluated relative to spatially distributed demand and the cost of reaching the facilities.

Two related topics:

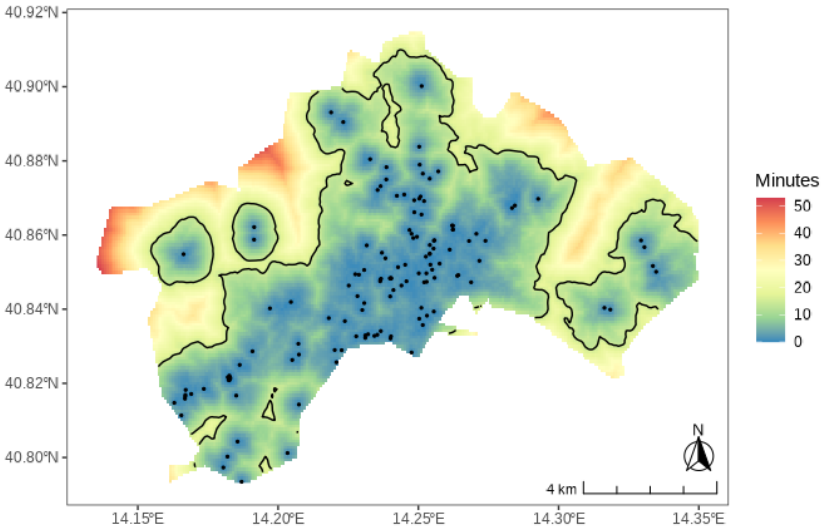
- **Location:** where should facilities be placed?
- **Allocation:** which demand locations are served by each facility?

Applications (optimization/evaluation of):

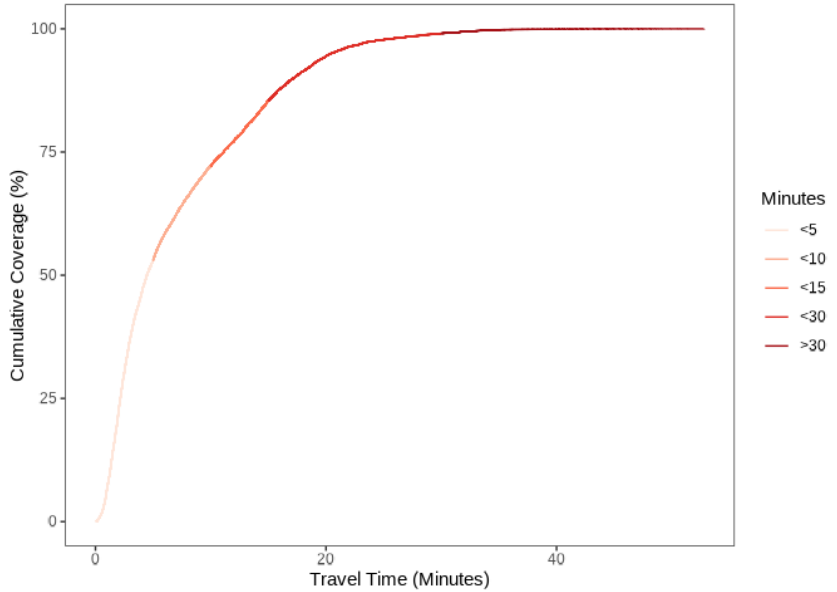
- healthcare, education, emergency facilities
- public transport stops, public EV chargers, bike share points
- warehouses & distribution hubs, collection points (waste management)

SSA: Location–allocation analysis example

Public drinking water fountains in Naples (Italy)



Map of accessibility to facility points (water fountains), with 15-minute isochrones highlighted.



Cumulative coverage plot (population density-based, 85.45% of coverage within the 15 minutes threshold).

Example based on the `locationallocation` package
`https://github.com/giacfalk/locationallocation`

SSA: Location–allocation analysis

Let:

$I = \{1, \ldots, m\}$ Set of candidate facility locations.

$i \in I$ Index identifying a candidate facility location.

$m = |I|$ Number of candidate facility locations.

$J = \{1, \ldots, n\}$ Set of spatial demand points.

$j \in J$ Index identifying a demand point.

$n = |J|$ Number of demand points.

$w_j \geq 0$ Demand weight at demand point j.

$d_{ij} \geq 0$ Travel time from candidate facility i to demand point j.

$T > 0$ Maximum acceptable travel-time threshold for coverage.

$N_j \subseteq I$ Set of candidate facilities capable of covering demand point j:

$$N_j = \{i \in I : d_{ij} \leq T\}.$$

$P^* \in \{1, \ldots, m\}$ Maximum number of new facilities that may be selected (facility-constrained discrete problems).

SSA: Location–allocation analysis

Maximal Coverage Location Problem (MCLP), two sets of binary variables:

$$x_i \in \{0,1\}, \quad \forall i \in I,$$

$$\text{where } x_i = \begin{cases} 1, & \text{if a facility is selected at candidate location } i, \\ 0, & \text{otherwise.} \end{cases}$$

All facilities assumed equal. The value $x_i = 1$ means that candidate facility location i is selected, it does not address the supply capacity of facilities, (corresponding problems discussed separately).

$$y_j \in \{0,1\}, \quad \forall j \in J,$$

$$\text{where } y_j = \begin{cases} 1, & \text{if demand point } j \text{ is covered,} \\ 0, & \text{otherwise.} \end{cases}$$

A value $y_j = 1$ means that demand point j is classified as covered by at least one selected candidate within threshold T.

SSA: Location–allocation analysis

Objective: maximize covered weighted demand

$$\max_{\boldsymbol{x}} \quad \sum_{j \in J} w_j y_j.$$

Coverage constraints

$$y_j \leq \sum_{i \in N_j} x_i, \qquad \forall j \in J.$$

Demand point j may be marked as covered only if at least one candidate in its coverage set N_j is selected. If one or more covering candidates x_i are selected, the objective function favors setting $y_j = 1$ whenever $w_j > 0$.

Facility-number constraint can be implemented

$$\sum_{i \in I} x_i \leq P^*.$$

Binary restrictions

$$x_i \in \{0, 1\}, \quad \forall i \in I, \qquad y_j \in \{0, 1\}, \quad \forall j \in J.$$

SSA: Location–allocation analysis

The total amount of spatial demand is:

$$W_{\text{total}} = \sum_{j \in J} w_j.$$

The amount of covered demand is (maximization objective):

$$W_{\text{covered}} = \sum_{j \in J} w_j y_j.$$

The weighted coverage share is therefore:

$$C = \frac{W_{\text{covered}}}{W_{\text{total}}} = \frac{\sum_{j \in J} w_j y_j}{\sum_{j \in J} w_j}.$$

Provided that: $w_j \geq 0$ and $\sum_{j \in J} w_j > 0$, we have: $0 \leq C \leq 1$.

As total demand is fixed, maximizing W_{covered} is equivalent to maximizing C.

Greedy algorithm for MCLP:

Let:

$$S_0 = \varnothing$$

be the initial selected-facility set, and let:

$$J_0^{\text{unmet}} = J$$

be the initial set of uncovered demand points. At iteration k, select:

$$i_k^* = \arg\max_{i \in I \setminus S_{k-1}} \sum_{j \in J_{k-1}^{\text{unmet}}} w_j \, \mathbf{1}(d_{ij} \leq T).$$

Update the selected facilities:

$$S_k = S_{k-1} \cup \{i_k^*\}.$$

Update unmet demand:

$$J_k^{\text{unmet}} = J_{k-1}^{\text{unmet}} \setminus \left\{ j : d_{i_k^* j} \leq T \right\}.$$

$\rightarrow$ From the set of previously $(k-1)$ unmet demand points j, we remove those that are within threshold T of the newly selected facility i_k^*.

Existing and new facilities: coverage by existing facilities:

Let F denote the set of existing facilities, indexed by $h \in F$.

Let $d_{hj}^F \geq 0$ be travel time from existing facility h to demand point j.

For location j, the nearest-facility travel time is: $a_j(F) = \min_{h \in F} d_{hj}^F$.

For location j, the existing-facility coverage is: $y_j^F(T) = \begin{cases} 1, & \text{if } a_j(F) \leq T, \\ 0, & \text{otherwise.} \end{cases}$

The existing coverage share is: $C(F,T) = \dfrac{\sum_{j \in J} w_j y_j^F(T)}{\sum_{j \in J} w_j}$.

Coverage by existing and newly allocated facilities:
Greedy algorithm can be applied, we start from F instead of an empty set.

Run the algorithm & define the newly selected set: $S = \{i \in I : x_i = 1\}$.

The complete facility system now contains: $F \cup S$.

For demand point j, nearest-facility travel time becomes:

$$a_j(F, S) = \min \left\{ \min_{h \in F} d^F_{hj},\ \min_{i \in S} d_{ij} \right\}.$$

Demand point j is covered $(y_j = 1)$ when: $a_j(F, S) \leq T$.

The resulting coverage share is: $C(F, S, T) = \dfrac{\sum_{j \in J} w_j \, \mathbf{1}[\, a_j(F, S) \leq T \,]}{\sum_{j \in J} w_j}$.

SSA: Location–allocation analysis

Continuous vs. discrete MCLP:

- In the continuous problem, potential facility locations are searched over the full spatial lattice (raster), representing the study area.
- In the discrete problem: $I = \{1, \ldots, m\}$ is a finite set of predefined candidate locations. A feasible selected set satisfies: $S \subseteq I, \quad |S| \leq P^*$.

Multiple (heuristic) approaches are possible. Optimization approach by `locationallocation` package:

- maximum uncovered demand weight;
- maximum kernel density of uncovered demand weights.

The heuristic procedures terminate when either of the following stopping criteria is satisfied:

- a specified number of facilities has been allocated,
- a specified demand-coverage target has been reached.

SSA: Alternative Location–Allocation Objectives

Let $S \subseteq I$ be a selected set satisfying:

$$|S| = P^*.$$

Define nearest-facility travel time:

$$a_j(S) = \min_{i \in S} d_{ij}.$$

Maximal coverage

$$\max_{|S|=P^*} \sum_{j \in J} w_j \mathbf{1}[a_j(S) \leq T]\,.$$

***p*-median**

$$\min_{|S|=P^*} \sum_{j \in J} w_j a_j(S).$$

***p*-center**

$$\min_{|S|=P^*} \max_{j \in J:\, w_j > 0} a_j(S).$$

Transportation-related applications

Transportation-related applications

Routing analysis: paths through a network, identification of routes satisfying a specified criterion.

A routing problem setup:

- a network of feasible connections,
- an origin and destination (1–1, 1–many, many–1, many–many),
- non-negative cost assigned to each connection,
- objective function (definition of target, preferences)
- restrictions if applicable (capacity, time-window, etc.).

Common routing objectives include:

- minimizing distance / travel time / generalized cost,
- visiting several destinations efficiently,
- routing multiple vehicles from a common depot.

Transportation-related applications

Graph representation of the transport network: $G = (V, E)$, with

- $v_i \in V = \{1, \ldots, n\}$; (vertices),
- $e = (i, j) \in E$; edge connecting vertices i and j,
- $c(i, j) = c_{ij} \geq 0$; cost of traversing edge (i, j),
- $s \in V$; origin vertex,
- $t \in V$ destination vertex.

Typically, graphs in transport networks are **directed** (permitted direction of travel matters), and **weighted** (edges have different traversal costs). For directed weighted graphs, the following is possible:

$$(i, j) \in E \qquad \text{but} \qquad (j, i) \notin E$$

or

$$c_{ij} \neq c_{ji}.$$

Path & path cost

Path from origin s to destination t is an ordered sequence:

$$P_{st} = (v_0, v_1, \ldots, v_r),$$

where: $v_0 = s,\ v_r = t$, and every consecutive pair of vertices forms an edge:

$$(v_{\ell-1}, v_\ell) \in E, \qquad \ell = 1, \ldots, r.$$

The total cost of path P_{st} is: $C(P_{st}) = \sum_{\ell=1}^{r} c(\ell - 1, \ell)\,.$

- In an unweighted graph, path cost may be represented by r ($c_{ij} = 1\ \forall i, j$; and the number of traversed edges is r).
- Weighted graphs: a path with fewer edges may be longer in physical distance. It is possible that:

$$r(P^{(1)}) < r(P^{(2)}), \quad \text{while}$$
$$C(P^{(1)}) \geq C(P^{(2)})$$

Shortest-path

Let

$\mathcal{P}_{st}$ be a set of all feasible paths connecting s and t,

P^*_{st} a path from s to t with minimum cost (minimum value of the chosen path-cost function, not necessarily physical distance),

$d_G(s,t)$ minimum attainable cost for a path between s and t (graph G).

The shortest-path problem: $$P^*_{st} \in \arg\min_{P \in \mathcal{P}_{st}} C(P).$$

The argmin function returns the set of paths attaining the minimum cost (shortesth path). Distinct paths may attain the same minimum cost, i.e.:

$$P^{*(1)}_{st} \neq P^{*(2)}_{st}, \qquad C(P^{*(1)}_{st}) = C(P^{*(2)}_{st}).$$

The corresponding shortest-path distance (on a graph G) is:

$$d_G(s,t) = \min_{P \in \mathcal{P}_{st}} C(P).$$

Transportation-related applications: Dijkstra

Dijkstra's Shortest-Path Algorithm finds minimum-cost paths from one origin s to all reachable vertices when:

$$c_{ij} \geq 0 \qquad \forall (i,j) \in E.$$

For each vertex $v \in V$, maintain a tentative distance label:

$$D(v).$$

Initialization:

$$D(s) = 0, \qquad D(v) = \infty \quad \forall v \neq s.$$

When the unsettled vertex u with the smallest tentative label is selected, each outgoing edge (u, v) is relaxed:

$$D(v) \leftarrow \min \{D(v),\ D(u) + c_{uv}\}.$$

Once u is settled, $D(u)$ equals the shortest-path distance from s to u.

Transportation-related applications

Shortest-Path-Based Routing Tasks Let $s \in V$ be a selected origin and let $\tau > 0$ be a maximum acceptable route cost. **Network service area**

$$R_s(\tau) = \{v \in V : d_G(s, v) \leq \tau\}.$$

This identifies all vertices reachable from s within cost τ. Other

shortest-path-based tasks include:

- **isochrones:** areas reachable within a travel-time limit;
- **isodistances:** areas reachable within a distance limit;
- **alternative routes:** several low-cost routes between the same endpoints;
- **restricted routing:** routes excluding specified edges, locations, or transport modes;
- **multimodal routing:** routes combining several transport networks;
- **origin–destination matrices:** shortest-path costs among multiple locations.

Transportation-related applications: TSP

Traveling Salesman Problem Let:

$$U = \{u_1, \ldots, u_q\} \subseteq V$$

be a set of q required visit locations. Define the pairwise shortest-path cost:

$$a_{rs} = d_G(u_r, u_s), \qquad u_r, u_s \in U.$$

A tour is an ordering represented by a permutation:

$$\pi = (\pi_1, \ldots, \pi_q)$$

of the indices $\{1, \ldots, q\}$. The Traveling Salesman Problem seeks:

$$\pi^* \in \arg\min_{\pi \in \Pi_q} \left[\sum_{r=1}^{q-1} a_{\pi_r \pi_{r+1}} + a_{\pi_q \pi_1} \right],$$

where Π_q is the set of all permutations of the q visit locations.

Transportation-related applications

Extensions of the Traveling Salesman Problem The Traveling Salesman Problem assumes one traveler or vehicle and one closed tour visiting all required locations. Important extensions include:

Open TSP The route does not return to its starting location.

Multiple TSP Several travelers jointly visit the required locations.

Vehicle Routing Problem Several vehicle routes begin and usually end at a depot.

Capacitated VRP Each vehicle has capacity $Q > 0$, while customer u_r has demand $q_r \geq 0$.

VRP with time windows Customer u_r must be visited during an admissible interval $[a_r, b_r]$.

Pickup-and-delivery routing Specified pickup locations must precede their corresponding delivery locations.